

Empirical Analysis of Transformers on Graph Connectivity

Part VII: Opening the Trunk — Why Does an Unbalanced Two-Chain Generalise?

Edoardo Paccagnella

July 13, 2026

Abstract

Report VI (Thread B) found a puzzle it could not explain: the same trained $L=2$ transformer, same weights, same canvas size $n=40$, solves a two-chain graph split into paths of length a and $n-a$ perfectly when a is small (e.g. $4+36$, correct out to the far end of the long path) but fails completely once the split is close to balanced (e.g. $17+23$, $20+20$ — correct only up to the architectural distance wall $3^L=9$, then exactly zero). Every report so far has read the model *behaviourally*: from its outputs, not from what it computes. This report opens the trunk — the attention weights, the read-out matrix, the per-layer embeddings — and asks what is mechanistically different between a split the model solves and one it does not.

1 Introduction

1.1 What Reports I–VI established

We treat the following as established (documented, multi-seed, in Reports I–VI); the new investigation must stay consistent with them.

- **There is a sharp, architectural distance wall at 3^L ($= 9$ at $L = 2$),** which moves with depth and appears even on graph families seen $\sim 10^8$ times in training (Reports III–IV).
- **The model computes distance-bounded matrix powering, not a bounded traversal:** a DFS/BFS reading is rejected on constructed and (partly) on natural graphs; a soft, learned “node budget” on dense structure is a removable data prior, not a second capacity (Report V).
- **Parallel routes and repeated local structure both help, within capacity, in ways that are data-shaped rather than architecturally free:** more disjoint routes between two endpoints rescue a connection the model could not otherwise confirm; a learned local rule (a bridge between two cliques) composes only weakly along a chain (Report VI).
- **The unexplained result this report targets:** sweeping the split $(a, n-a)$ of a two-chain graph at fixed $n = 40$, the same checkpoint goes from solving *every* pair in the long path (out to ~ 38 hops) to solving *none* beyond the 9-hop wall, as a crosses a threshold close to the capacity itself (Report VI, Thread B). No account of the mechanism was given.
- **Every report through Report VI is behavioural.** Model outputs are compared against oracles, ablated data distributions, or read-out variants — never against the attention weights or the read-out matrix themselves. `model.py` exposes tools for exactly that (real per-layer attention weights, per-layer embeddings) that no report has yet used.

1.2 The new question: what is mechanistically different?

Report VI’s own closing paragraph names this split puzzle as the one open thread it could reproduce but not explain. This report is the direct follow-up: instead of another behavioural sweep, we open the trained model at the level of its attention scores and its read-out weights, on the exact same checkpoints, and ask what changes between a split the model solves and one it does not.

1.3 Three candidate hypotheses

We state candidate explanations up front, so each experiment is a test that could come out either way — not a search for evidence of an assumed answer.

1. **Genuine propagation.** The model actually traces distance along the path; behaviour should be robust to relabelling nodes, to where the short component sits, and to the internal edge order of the long path.
2. **Short-component-resolved default.** The model fully resolves whichever component is small enough to fit inside its capacity (`short_len` $\lesssim 9$) and defaults every other pair to “connected” — a shortcut that is correct whenever exactly one component is small, and has nothing to fall back on once *both* components are too large to resolve.
3. **Label/order shortcut.** The model exploits some regularity in how node indices map to structure, rather than the abstract graph. (The existing evaluation already relabels every test graph independently at random before scoring it, which substantially weakens the crude form of this hypothesis before we even start; we still check it directly against the read-in/read-out weights.)

2 Setup

Model. The same base model as Reports III–VI: the RoBERTa-faithful standard transformer with the *linear* read-out, $L = 2$, single head, $d_{\text{model}} = 512$, normalised-ReLU attention $\alpha = (1/n) \text{ReLU}(QK^\top/\sqrt{d_h})$, trained online at $n = 40$ on a single clean distribution of disjoint paths (unions of 1–4 paths covering all n nodes), four seeds. This is the exact checkpoint set behind Report VI’s Thread B.

The forward pass, in full. Because §4.7–§4.11 open the trunk directly, we fix here, once, exactly what the model computes at inference, so that every quantity used later — the attention score, the normalised-ReLU weights, the message contribution, the node-to-node contribution — has a precise definition. The input is the self-loop-augmented adjacency $A' = A + I \in \{0, 1\}^{n \times n}$. The read-in, once,

$$H^{(0)} = \text{LayerNorm}(A' W_{\text{in}} + \mathbf{1} b_{\text{in}}^\top),$$

starts row i , before the LayerNorm, as simply the sum of one learned vector per neighbour of i (including i itself, via the self-loop) plus a bias — no attention yet. Each of the two transformer

blocks (layer $\ell = 1, 2$, post-LayerNorm) then maps $H^{(\ell-1)}$ to $H^{(\ell)}$ by

$$\begin{aligned} Q_\ell &= H^{(\ell-1)} W_Q^{(\ell)}, \quad K_\ell = H^{(\ell-1)} W_K^{(\ell)}, \quad V_\ell = H^{(\ell-1)} W_V^{(\ell)} \\ S_\ell &= Q_\ell K_\ell^\top / \sqrt{d_h} \\ \alpha_\ell &= \frac{1}{n} \text{ReLU}(S_\ell) \\ \text{AttnOut}_\ell &= (\alpha_\ell V_\ell) W_O^{(\ell)} \\ \hat{H}_\ell &= \text{LayerNorm}(H^{(\ell-1)} + \text{AttnOut}_\ell) \\ H^{(\ell)} &= \text{LayerNorm}(\hat{H}_\ell + \text{FFN}^{(\ell)}(\hat{H}_\ell)) \end{aligned}$$

Here $d_h = d_{\text{model}}$ (single head); S_ℓ is the raw attention score; α_ℓ is the normalised-ReLU attention — *not* a softmax, it does not sum to 1 per row; α_ℓ weights the *values* V_ℓ , and $W_O^{(\ell)}$ is the output projection applied afterwards; $\hat{H}_\ell$ adds that attention output back onto the residual stream; $H^{(\ell)}$ then adds the feed-forward step $\text{FFN}^{(\ell)}$ on top, again as a residual. Layer 2 repeats this starting from $H^{(1)}$, with its *own* W_Q, W_K, W_V, W_O and its own feed-forward weights — it does not reuse α_1 in any way; the two layers are two separate passes of this same recipe, run one after the other. The read-out (§2 below) is applied once, at the very end, to $H^{(2)}$.

The same computation, one number at a time. Every equation above is a matrix equation; every one of them is also just an ordinary sum over the $d := d_{\text{model}} = 512$ hidden features, and it is worth writing out explicitly, matching the actual implementation entry by entry rather than only in matrix form. Two building blocks recur throughout and are defined once, here. A linear layer with weight matrix W (indexed input-feature, output-feature) and bias b maps a row x to

$$y_r = \sum_s x_s W_{s,r} + b_r;$$

every one of $W_{\text{in}}, W_Q^{(\ell)}, W_K^{(\ell)}, W_V^{(\ell)}, W_O^{(\ell)}, W_1^{(\ell)}, W_2^{(\ell)}$ in this report is one such matrix, each with its own bias — all dropped from the matrix equations above only to keep them short, never dropped from the actual computation. **LayerNorm** is applied independently to each *node's* own row $x \in \mathbb{R}^d$, with its own learned per-feature scale γ and shift β :

$$\text{LayerNorm}(x)_r = \gamma_r \frac{x_r - \bar{x}}{\sqrt{\text{Var}(x) + \epsilon}} + \beta_r, \quad \bar{x} = \frac{1}{d} \sum_s x_s, \quad \text{Var}(x) = \frac{1}{d} \sum_s (x_s - \bar{x})^2;$$

crucially $\bar{x}$ and $\text{Var}(x)$ come from that *one* node's own d numbers, never averaged across nodes, and there are five independent LayerNorms in the trunk (the embedding one, plus one after each residual, in each of the two layers), each with its own γ, β . $\text{GELU}(x) = x \Phi(x)$ applies elementwise, Φ the standard normal CDF.

With these two building blocks, every quantity in this report is, entry by entry: the read-in,

$$u_{i,r} = \sum_{k=1}^n A'_{ik} (W_{\text{in}})_{k,r} + (b_{\text{in}})_r, \quad h_{i,r}^{(0)} = \text{LayerNorm}(u_{i,r});$$

the query at layer ℓ (identically for $k_{i,r}$ and $v_{i,r}$, with their own weights),

$$q_{i,r} = \sum_{s=1}^d h_{i,s}^{(\ell-1)} (W_Q^{(\ell)})_{s,r} + (b_Q^{(\ell)})_r;$$

the attention score and weight for the *ordered pair* (i, j) — a single number, not a matrix, one for every pair of nodes,

$$S_{\ell,ij} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} q_{i,r} k_{j,r}, \quad \alpha_{\ell,ij} = \frac{1}{n} \max(0, S_{\ell,ij});$$

the attention output — first mix *values* across every other node j , feature by feature, then project the mixed result,

$$m_{\ell,i,s} = \sum_{j=1}^n \alpha_{\ell,ij} v_{j,s}, \quad \text{AttnOut}_{\ell,i,r} = \sum_{s=1}^d m_{\ell,i,s} (W_O^{(\ell)})_{s,r} + (b_O^{(\ell)})_r;$$

the first residual and LayerNorm,

$$\hat{h}_{\ell,i,r} = \text{LayerNorm}(h_{i,\cdot}^{(\ell-1)} + \text{AttnOut}_{\ell,i,\cdot})_r;$$

the feed-forward step (hidden width $d_{\text{ff}} = 2048$, GELU in between the two linear maps),

$$u_{\ell,i,m}^{\text{ff}} = \sum_{s=1}^d \hat{h}_{\ell,i,s} (W_1^{(\ell)})_{s,m} + (b_1^{(\ell)})_m, \quad g_{\ell,i,m} = \text{GELU}(u_{\ell,i,m}^{\text{ff}}),$$

$$\text{FFN}^{(\ell)}(\hat{h}_{\ell,i})_r = \sum_{m=1}^{d_{\text{ff}}} g_{\ell,i,m} (W_2^{(\ell)})_{m,r} + (b_2^{(\ell)})_r;$$

and the second residual and LayerNorm, closing the layer,

$$h_{i,r}^{(\ell)} = \text{LayerNorm}(\hat{h}_{\ell,i,\cdot} + \text{FFN}^{(\ell)}(\hat{h}_{\ell,i}))_r.$$

Every sum above runs over an ordinary feature index (r, s, m) ; the *only* place a sum ever runs over all n nodes at once, rather than over d or d_{ff} features, is the value-mixing step $m_{\ell,i,s} = \sum_j \alpha_{\ell,ij} v_{j,s}$ — the one place information actually moves between nodes. The same pattern gives the read-out logit at the level of a single pair (§2 below): $z_{ij} = \sum_{r=1}^d h_{i,r}^{(2)} (W_{\text{out}})_{r,j} + b_j$.

Message contribution and node-to-node contribution: what they measure, and how they are computed. Two derived quantities recur in §4.7 and §4.11. The **message contribution** $\alpha_{ij} \|W_O v_j\|$ uses the actual value vector v_j and output projection W_O of one layer to size the additive message node j sends into node i ’s residual stream *at that layer* — it is exact for that one layer’s attention branch, but does not include the residual stream itself or the feed-forward step, both of which also update h_i . The **node-to-node contribution**

$$C_{ik} = \left\| \frac{\partial h_i^{(2)}}{\partial h_k^{(0)}} \right\|_F$$

answers the question directly, across *both* layers at once: how much does node i ’s final embedding move if node k ’s read-in embedding moves, holding everything else fixed? We compute it by literal automatic differentiation through the exact forward pass above — the real V, W_O , the real residual identity, the real feed-forward step, and LayerNorm’s true local derivative are all part of the computation, none of them dropped or approximated; the only sense in which this is “local” is the ordinary one for any derivative, that it is evaluated at one specific graph’s actual activations.

Why the read-out shape matters. With the linear read-out, the logit for pair (i, j) is $z_{ij} = h_i^\top w_j + b_j$, where w_j is the j -th row of the read-out matrix W_{out} — a fixed, learned vector attached to the *target* node j , not derived from j ’s own embedding. Node i ’s embedding h_i alone must therefore carry enough information to answer, for *every* other node j , “are we connected” — all the structure the model has learned about node j lives in w_j , fixed across every graph. This is why the read-out matrix itself, not only the attention weights, is a first-class object of study here.

Test graphs. Two-chain graphs partitioning all $n = 40$ nodes into two disjoint paths of size a and $n - a$, swept over the *full* range $a = 1, \dots, 20$ (Report VI reported only a handful of points). A new three-component construction — one small, resolvable path plus two large, individually-unresolvable paths that are *not* connected to each other — is used as a falsification probe for hypothesis 2 (§4.9).

3 Plan of experiments

Behavioural groundwork (§4.1–4.3). A dense split sweep and an explicit random-relabelling check, to fix the shape of the puzzle precisely and close off the crude label-shortcut hypothesis before opening the trunk.

Read-out mechanism (§4.4–4.5). The $h_i^\top w_j$ decomposition and the geometry of W_{out} ’s rows — does the read-out encode component membership as a blob, or a graded function of distance?

Attention mechanism (§4.7). Real per-layer attention weights, the exact node-to-node contribution, and the message contribution each node actually sends — does attention itself reach differently in a solved split than in a failed one?

Falsification test (§4.9). The three-component construction: if the model is really defaulting “not in the small component” to connected, it should wrongly merge two large components that are genuinely disconnected from each other.

A second training distribution, and a third canvas size. Every one of the four threads above is repeated, at the same $n = 40$ canvas, on a model trained purely on sparse Erdős–Rényi graphs — which never contains a disconnected or multi-path graph in training — immediately after its disjoint-paths counterpart, so the two conditions can be read side by side. §4.15 then moves to a larger, sparser canvas ($n = 64$); §4.16 closes the sweep at a smaller one ($n = 20$, where the architectural capacity can no longer be exceeded by any split), on both training distributions.

Why the read-out itself is also worth varying. Every result up to this point uses the linear read-out, $z_{ij} = h_i^\top w_j + b_j$ (§2). This has a specific architectural consequence worth stating plainly: w_j is a single vector, fixed once training ends, that must work as a classifier for node j against *every* possible query embedding h_i — it is not itself a function of j ’s own embedding, only of j ’s identity as a target. A finding built on this read-out could, in principle, be a property of that specific asymmetric construction rather than of how the trunk represents connectivity in general. The similarity read-out already used for other purposes in Report IV, $z_{ij} = \text{scale} \cdot \cos(h_i, h_j) + \text{bias}$, is a genuine function of *both* endpoints’ embeddings and carries no such fixed per-target vector, so repeating this report’s full battery on it is a direct test of whether the two-component completion signal is an artefact of the linear read-out’s shape or a property of the trunk that survives a structurally different, more symmetric read-out. §4.17 reports the result, on both training distributions, at the same $n = 40$ canvas.

4 Results

4.1 The split sweep, in full

Question. Report VI reported exact match at seven splits. Sweeping every $a = 1, \dots, 20$ locates the break precisely and lets us compare the model’s aggregate behaviour against a

genuine distance-bounded oracle.

Result: the break sits one node above the capacity, and beyond it the model’s aggregate behaviour matches pure distance-bounded matrix powering almost exactly. Figure 1 (top) confirms Report VI’s shape at full resolution: exact match is ≈ 1.0 for $a \leq 7$ (with ordinary sampling noise at $a = 3, 6$: 0.80–0.87, consistent across all four seeds, not a single bad run), collapses to exactly 0 from $a = 8$ on, and the long-component reach follows the same break with a soft floor-plus-recovery from $a \approx 11$ to $a = 20$ — the familiar shape of every beyond-capacity curve in this project. The new observation is the dashed and dotted curves: the model’s overall predicted-positive rate (fraction of all pairs called “connected”) tracks the *true* positive rate closely while $a \leq 7$, then drops sharply and, from $a \approx 11$ on, sits almost exactly on top of the positive rate a pure distance- ≤ 9 oracle would produce on the same graphs (0.36 measured vs. 0.35 from the oracle, flat in a because a long path’s 9-hop neighbourhood size does not depend on how far away the path’s other end is). **Once both components are individually beyond capacity, the model’s aggregate behaviour is indistinguishable from plain distance-bounded matrix powering;** the extra “complete the whole long component” behaviour is confined to the regime where one side is small.

split ($a, 40-a$)	exact	reach (long)	reach (short)	cut	pred. positive rate
(1, 39)	1.000	1.000	—	1.000	0.951
(4, 36)	1.000	1.000	1.000	1.000	0.820
(7, 33)	0.779	1.000	1.000	0.998	0.712
(8, 32)	0.000	0.992	1.000	0.925	0.699
(9, 31)	0.000	0.873	1.000	0.938	0.599
(10, 30)	0.000	0.648	1.000	0.999	0.434
(14, 26)	0.000	0.581	0.889	1.000	0.362
(17, 23)	0.000	0.638	0.792	1.000	0.362
(20, 20)	0.000	0.709	0.709	1.000	0.362

Table 1: *Model*: base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on a single clean distribution of disjoint paths, four seeds, $n = 40$ (the checkpoints behind Report VI’s Thread B). *Test set*: two-chain graphs split ($a, 40-a$), 200 freshly sampled, independently node-relabelled graphs per split per seed (800 total per split). *Metrics* (mean over seeds): whole-graph exact match; pairwise reach inside the long/short path (target connected; “—” at $a = 1$, no internal short-path pairs); pairwise accuracy across the two paths (target disconnected); the model’s overall fraction of pairs predicted connected. Full sweep $a = 1, \dots, 20$ in Figure 1.

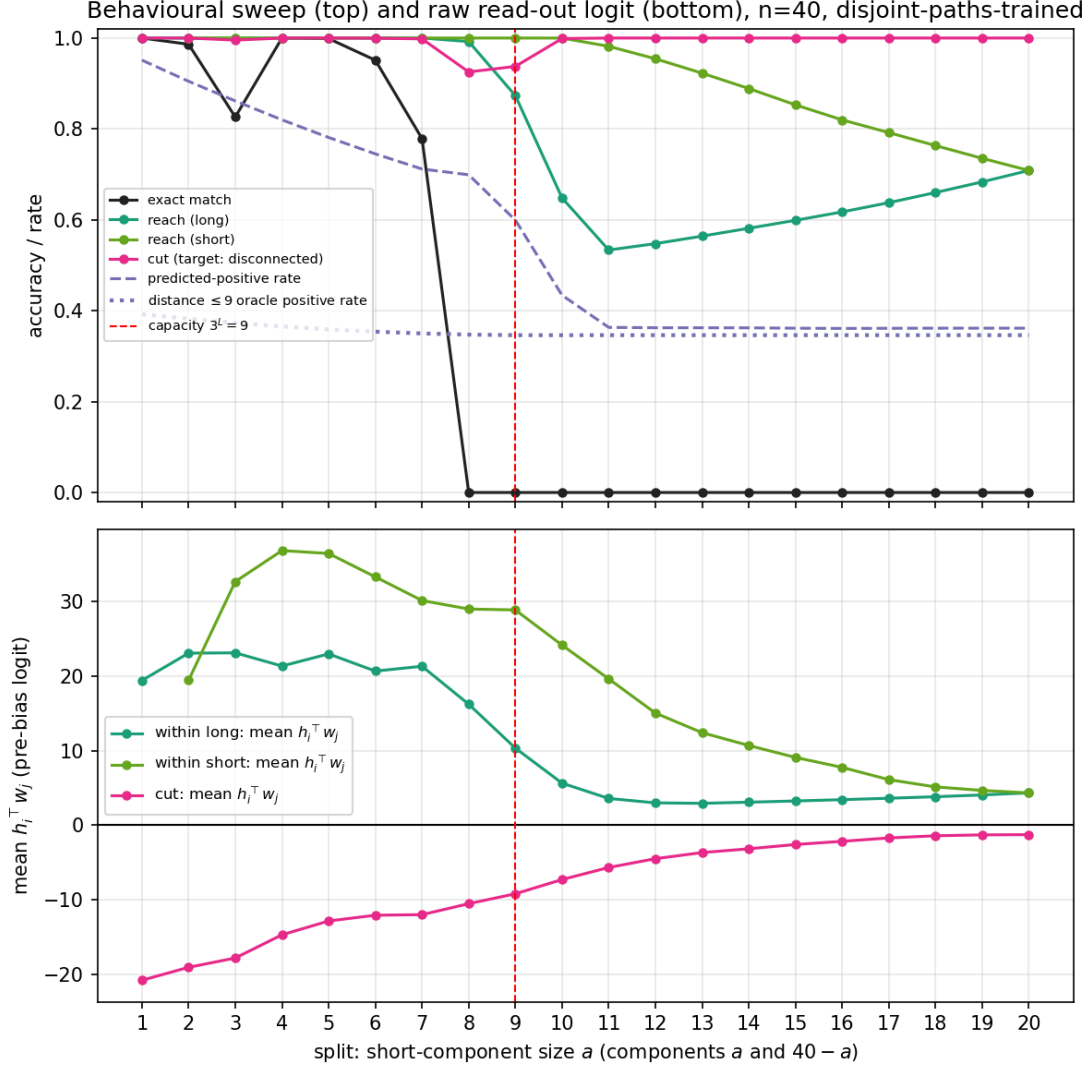


Figure 1: *Top: Model/training/test as in Table 1, full split sweep $a = 1, \dots, 20$; the dashed line is the model’s overall predicted- positive rate, the dotted line the positive rate a pure distance- ≤ 9 oracle would give on the identical graphs (both computed in closed form from the two path lengths, independent of any model). Bottom: same model and test graphs, the mean raw read-out logit $h_i^\top w_j$ (before adding the target’s bias b_j ; §2) for pairs within the long path, within the short path, and across the two paths, mean over seeds. Red dashed line: capacity $3^L = 9$.*

The collapse is not a discontinuity in the mechanism. Figure 1 (bottom) is the reason the exact-match curve looks like a cliff: the raw, pre-bias logit $h_i^\top w_j$ for a pair inside the long path is not a switch, it is a smoothly decaying quantity. It sits saturated and strongly positive for $a \leq 7$ (≈ 17 – 36), falls through zero across $a = 8$ – 11 (exactly where accuracy collapses), and settles into a small, slowly *recovering* positive value from $a \approx 11$ to $a = 20$ — the same soft floor-plus-recovery shape as the accuracy curve above it, now visible in the model’s raw confidence rather than only in its thresholded predictions. The cut logit (across the two paths, target disconnected) tells a different story: it starts very negative (≈ -21 at $a = 1$) and its magnitude shrinks steadily as the split balances, but it *never crosses zero* at any split tested — the “different component” decision degrades in confidence but not in sign, consistent with cut accuracy staying at ≈ 1.0 across the whole sweep in Table 1.

4.2 The same sweep on an ER-trained model: the failure mode mirrors, it does not repeat

Question. Every result above is on a model trained on disjoint paths. Does a model that has never seen a two-chain, or any multi-component graph, in training show the same completion signal on the identical test sweep — or does it fail a different way?

Result: reach barely degrades at any split, but cut collapses steadily as the split balances — the mirror image of the disjoint-paths result. Table 2 gives the same sweep on four checkpoints trained purely on sparse Erdős–Rényi graphs ($n = 40$, never a two-chain or any disconnected graph in training). The shape is entirely different from Table 1. Reach in the long component never really drops: it sits at 0.90–0.98 across the *whole* sweep, with no floor-and-recovery dip in the middle — the model confirms far-apart pairs about as well at a balanced split as at an unbalanced one. What degrades instead is cut: from 1.000 at $a = 1$ it falls steadily to 0.15 by $a = 20$, the opposite of Table 1, where cut stayed at ≈ 1.0 throughout and reach was the one that collapsed. Exact match also breaks earlier and less cleanly than on the disjoint-paths model (already 0 by $a = 7$, where the disjoint-paths model was still 0.78), because even the unbalanced splits solved here are solved by high reach *and* a still-imperfect cut, not by a clean short-component resolution. The raw read-out logit (not tabulated) confirms the mechanism directly: the within-long logit stays modest throughout (3.8–6.5, never saturating the way the disjoint-paths model’s does at 17–36), while the cut logit — unlike the disjoint-paths model, where it never changed sign — starts very negative (-25.4 at $a = 1$) and *crosses zero* around $a \approx 14$, turning mildly positive by $a = 17$ – 20 : the model does not just lose confidence that a cross-pair is disconnected, for many such pairs it comes to believe the opposite.

split ($a, 40-a$)	exact	reach (long)	reach (short)	cut	pred. positive rate
(1, 39)	0.108	0.964	—	1.000	0.918
(4, 36)	0.719	0.999	1.000	0.999	0.820
(7, 33)	0.000	0.978	1.000	0.533	0.831
(8, 32)	0.000	0.947	1.000	0.556	0.789
(9, 31)	0.000	0.920	1.000	0.575	0.753
(10, 30)	0.000	0.899	1.000	0.523	0.749
(14, 26)	0.000	0.924	0.993	0.292	0.835
(17, 23)	0.000	0.944	0.977	0.192	0.885
(20, 20)	0.000	0.961	0.958	0.149	0.906

Table 2: *Model:* base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on pure sparse Erdős–Rényi (ER(40, 0.05), mean degree ≈ 2.0 ; never any disconnected or multi-path graph), four seeds, $n = 40$. *Test set/metrics as in Table 1.*

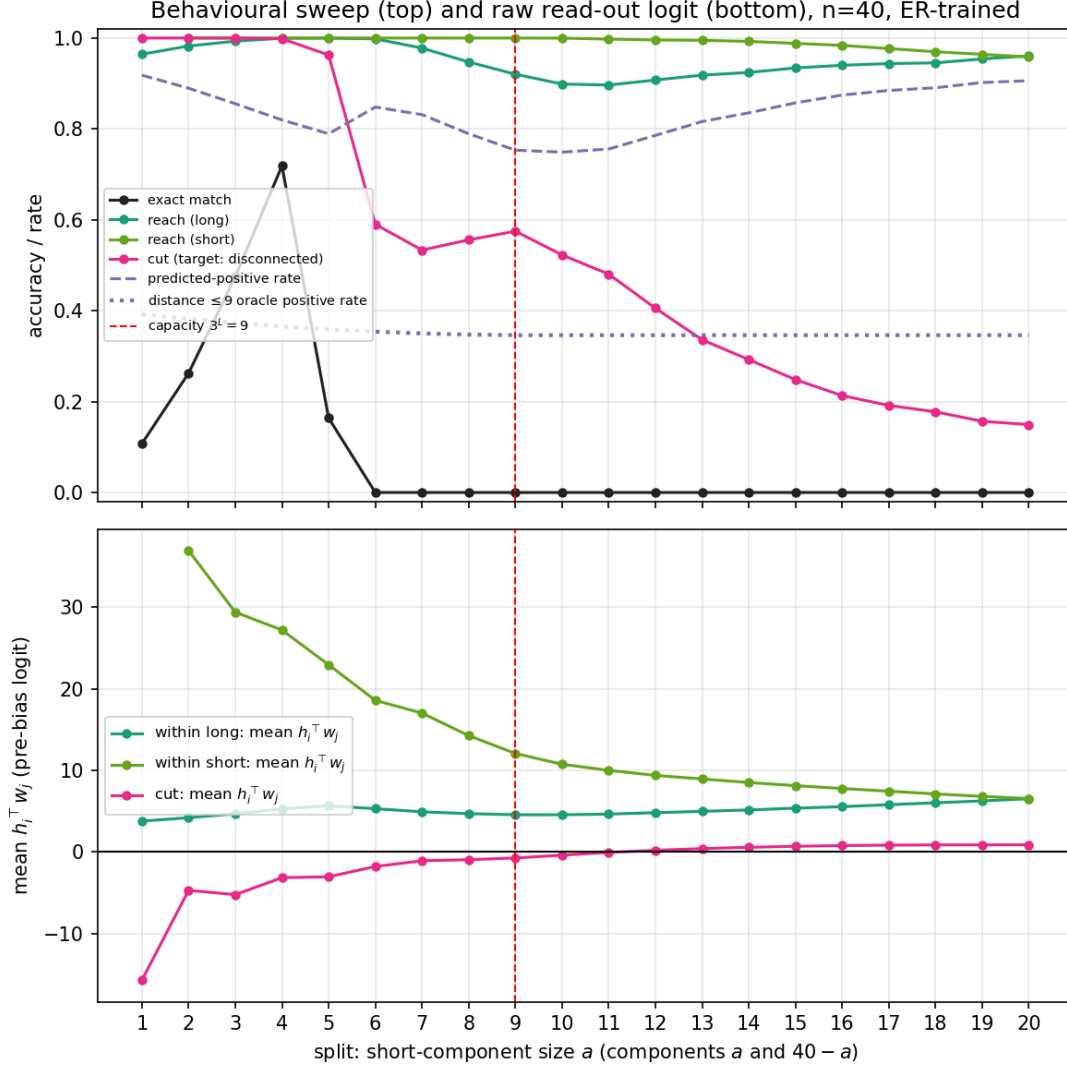


Figure 2: *Model/test/metrics as in Figure 1 but trained on pure sparse Erdős–Rényi (ER(40, 0.05), mean degree ≈ 2.0), four seeds, $n = 40$.*

4.3 Relabelling does not explain it

Every graph in §4.1 is already independently relabelled at random before being scored (a fresh random permutation per graph, unpermuted before comparing predictions) — this is the existing methodology, not a new control. We add one direct comparison: the same split sweep with node labels *fixed* to a canonical order (the short path always occupying the first a indices) instead of randomised. Table 3 shows no meaningful difference at any split. Hypothesis 3 in its crude form — the model exploiting a fixed mapping from node index to structure — is not what is happening here.

split (a , $40-a$)	exact, random relabelling	exact, fixed labelling
(1, 39)	1.000	1.000
(4, 36)	1.000	1.000
(7, 33)	0.779	0.750
(8, 32)	0.000	0.000
(10, 30)	0.000	0.000
(17, 23)	0.000	0.000
(20, 20)	0.000	0.000

Table 3: *Model/training as in Table 1. Test set:* two-chain graphs at the split shown, 200 graphs per seed, either independently relabelled at random per graph (the condition behind every other result in this report) or with node labels fixed to a canonical order (short path first). *Metric:* whole-graph exact match, mean over four seeds.

4.4 What the read-out logit actually responds to

§4.1 already showed the headline pattern (Figure 1, bottom): the within-long logit is large and saturated while the short component is resolvable, collapses through the transition, and settles at a small, recovering value that mirrors distance-bounded matrix powering. The within-short logit is the mirror image by construction (symmetric around $a = 20$, where the two logits meet exactly). One additional check confirms the decomposition is measuring the right quantity: aggregating $h_i^\top w_j + b_j > 0$ by pair type reproduces the behavioural reach numbers of Table 1 to within sampling noise at every split (e.g. 0.650 vs. 0.648 at $a = 10$, 0.709 vs. 0.709 at $a = 20$) — the two independently-computed views of the same model agree exactly, as they must.

4.5 The geometry of the read-out matrix

If the model relied on a fixed node-index shortcut (hypothesis 3), we would expect a target-specific classifier vector w_j to look unusually different from its neighbours, or the direct read-in-to-read-out skip connection $E_{\text{in}}W_{\text{out}}^\top$ (bypassing both transformer layers entirely) to already carry most of the signal. Neither holds (Table 4): the w_j row norms are uniform across all 40 targets (standard deviation $\approx 4\%$ of the mean, every seed), the pairwise cosine similarity between different w_j ’s is small (≈ 0.06), and the skip-connection alignment is small everywhere (diagonal ≈ 0.2 – 0.3 , off-diagonal ≈ 0) — two orders of magnitude below the $h_i^\top w_j$ values of 10–35 that actually decide a prediction (§4.4). Whatever produces the large, split-dependent logit swings is built by the two attention/feed-forward layers, not read directly off which node is whose neighbour.

	$\ w_j\ $ (mean, std)	mean $\cos(w_j, w_k)$ ($j \neq k$)	$E_{\text{in}}W_{\text{out}}^\top$	
			diagonal	off-diagonal
seed 1000	5.18 ± 0.23	+0.065	+0.31	-0.03 ± 0.17
seed 2000	5.21 ± 0.19	+0.063	+0.25	-0.06 ± 0.19
seed 3000	5.09 ± 0.19	+0.063	+0.26	-0.02 ± 0.17
seed 4000	5.12 ± 0.20	+0.061	+0.24	$+0.03 \pm 0.16$

Table 4: *Model:* the four disjoint-paths-trained checkpoints of Table 1 (no test graphs — these are static properties of the trained weights). *Metrics:* the read-out row norms $\|w_j\|$ (mean $\pm$ std. over the 40 target nodes); the mean pairwise cosine similarity between different rows of W_{out} ; and the read-in/read-out skip-connection alignment $E_{\text{in}}W_{\text{out}}^\top$ (row k of the read-in weight against row j of the read-out weight), split into its diagonal ($k = j$) and off-diagonal mean $\pm$ std.

4.6 The same read-out geometry check on the ER-trained model

The same static check on the ER-trained checkpoints of §4.2 gives the same answer (Table 5): row norms uniform across targets, small pairwise cosine similarity, and a skip-connection alignment an order of magnitude below the logit values that actually decide a prediction. A different training distribution, entirely different split-sweep behaviour (§4.2), and the same refutation of a fixed node-index shortcut — whatever each model does with the split, neither reads it off W_{out} or W_{in} directly.

	$\ w_j\ $ (mean, std)	mean $\cos(w_j, w_k)$ ($j \neq k$)	$E_{\text{in}} W_{\text{out}}^\top$	
			diagonal	off-diagonal
seed 1000	5.68 ± 0.16	+0.127	−0.52	-0.16 ± 0.33
seed 2000	5.64 ± 0.16	+0.121	−0.40	-0.13 ± 0.35
seed 3000	5.61 ± 0.17	+0.125	−0.32	-0.10 ± 0.33
seed 4000	5.67 ± 0.21	+0.126	−0.63	-0.19 ± 0.35

Table 5: *Model:* the four ER-trained checkpoints of Table 2 (no test graphs). *Metrics as in Table 4.*

4.7 Attention: where the model actually looks

Question. The exact node-to-node contribution $C_{ik} = \|\partial h_i^{(2)} / \partial h_k^{(0)}\|_F$ (§2) measures, for each query node, how much its final representation actually moves with each other node’s read-in embedding. Does this quantity reach differently in a split the model solves than in one it does not?

Result: the long component draws contribution from the short one at every split tested, even the ones the model solves perfectly — but far more once the split balances. For each long-component node we sum C_{ik} over k in its own (long) component and over k in the short component, and read the short share as a leak fraction. Figure 3 shows it is never zero: even at $a = 1$, where the model is exact on every graph, 5.8% of the long component’s contribution mass already traces back to the single short node. It grows from there, but *not* smoothly — it rises to 11% at $a = 4$, 21–24% at $a = 7$ –8, then spikes to 43% right at $a = 10$ (immediately past where exact match collapses to zero), dips back to 27% at $a = 14$, and climbs again to 31–36% by $a = 17$ –20. Each of these values is tight across the four seeds (a few percentage points of spread at most), so the spike at $a = 10$ is a reproducible feature of the curve, not seed noise — the mechanism does not de-confine itself gradually and monotonically as the split balances; it is already leaking a little at the splits the model gets perfectly right, and the leak grows unevenly, peaking sharply right where behaviour first breaks before settling into a broad elevated plateau. **The qualitative reading survives even though the shape is messier than a smooth curve would suggest: confinement to the query’s own component is a matter of degree everywhere, and it degrades substantially, if unevenly, as the split balances.**

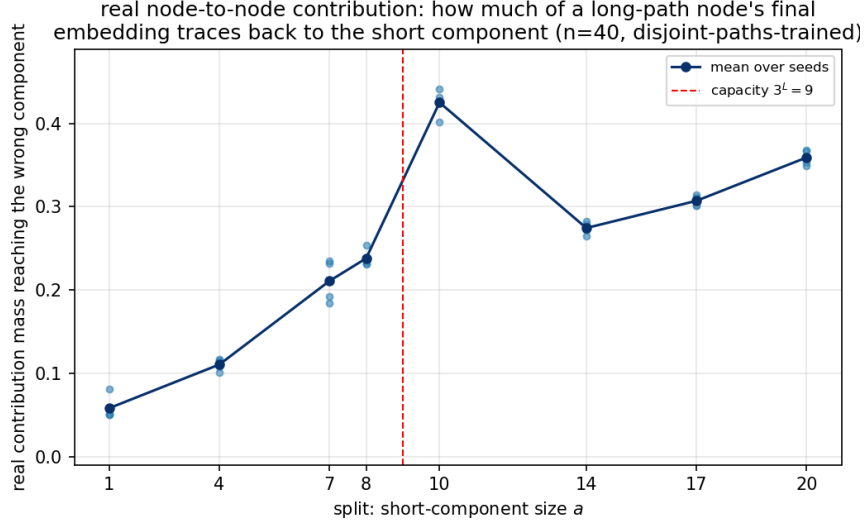


Figure 3: *Model/training as in Table 1. Test set:* two-chain graphs at the splits shown, 40 graphs per seed. *Metric:* for each long-component node, the fraction of its exact node-to-node contribution mass $C_{ik} = \|\partial h_i^{(2)} / \partial h_k^{(0)}\|_F$ (§2, real automatic differentiation through the whole forward pass) that traces back to the short component rather than its own long component, mean over long-component nodes; small dots are individual seeds, the line is the mean over seeds. Red dashed line: capacity $3^L = 9$.

4.8 The same leak fraction on the ER-trained model: smooth where the disjoint-paths one was not

The identical measurement on the ER-trained checkpoints (Figure 4) gives a strikingly *cleaner* curve than Figure 3: the leak fraction rises smoothly and monotonically, from exactly 0% at $a = 1$ (every one of the four seeds) through 5% ($a = 4$), 11–18% ($a = 7$ –10), to 21% ($a = 14$), 25% ($a = 17$) and 30% ($a = 20$) — no spike, no dip, a textbook gradual de-confinement as the split balances. The disjoint-paths model’s non-monotone spike at $a = 10$ (§4.7) is therefore not a generic property of how this leak fraction behaves near a capacity transition; it is specific to whatever the disjoint-paths training additionally builds into the completion mechanism. The ER model’s overall contribution magnitude is also far smaller throughout (consistent with its modest, never-saturating read-out logit, §4.2): the raw within-long contribution near the diagonal is ≈ 4 –4.5 at both $a = 4$ and $a = 20$, roughly an order of magnitude below the disjoint-paths model’s 53 at $a = 4$ (§4.12).



Figure 4: *Model/metric as in Figure 3 but trained on pure sparse Erdős–Rényi (ER(40, 0.05)), four seeds, $n = 40$. Test set: two-chain graphs at the splits shown, 64 graphs per seed.*

4.9 Falsification test: does the model actually merge two unrelated large components?

Question. If the model’s mechanism in §4.1–4.7 were simply “resolve whichever component is small, default everything else to connected” (hypothesis 2 in its crude form), then on a graph with *three* disjoint paths — one small and two large, the two large ones genuinely disconnected from each other — the model should wrongly call the two large components connected to each other (both read only as “not the small one”).

Result: the crude shortcut is refuted, but so is a full recovery — the completion trick is specific to a two-component canvas. It is not refuted by a coin flip: at every small-component size tested and in every one of the four seeds, the model correctly calls the two large components disconnected (cut = 1.000 throughout, Table 6) — the “which component” decision generalises cleanly to three components, not just two. But the within-large-component reach does *not* recover to the near-perfect values §4.1 found for a single long path beside a small stub: it sits at 0.71–0.86, matching almost exactly the degraded reach of a *balanced two-way* split of a similar size (Table 1: 0.71 at the balanced (20, 20) split). Adding a small, cleanly-resolvable third component does not rescue the two large components’ internal connectivity — each is treated as if it were the “long” side of a balanced two-way split, not as the sole *other* component in the graph. The completion trick found in §4.1 is specific to graphs with exactly two components, one of them small; it is not a general “not-small therefore connected” rule.

small	large ₁	large ₂	reach large ₁	reach large ₂	cut(S,L ₁)	cut(S,L ₂)	cut(L ₁ ,L ₂)
1	19	20	0.737	0.710	1.000	1.000	1.000
2	19	19	0.737	0.737	1.000	1.000	1.000
4	18	18	0.765	0.765	1.000	1.000	1.000
7	16	17	0.825	0.794	1.000	1.000	1.000
8	16	16	0.824	0.825	1.000	1.000	1.000
10	15	15	0.855	0.855	1.000	1.000	1.000

Table 6: *Model/training as in Table 1. Test set:* three disjoint paths partitioning all 40 nodes — one small path (size shown) and two large paths (sizes large₁, large₂), the two large paths *not* connected to each other; 300 graphs per seed, independently node-relabelled. *Metrics* (mean over four seeds, tight in every cell): pairwise reach inside each large path (target connected); pairwise accuracy between the small path and large₁/large₂, and between the two large paths (all three target disconnected — “cut(L₁,L₂)” is the decisive column).

4.10 The same falsification test on the ER-trained model: the crude shortcut is *not* refuted

Result: at $n = 40$, an ER-trained model *does* show the crude “not the small one” merge the disjoint-paths model refuted. Table 7 is the opposite pattern from Table 6. At a very small third component (small = 1, 2) the two large, mutually disconnected components are wrongly called connected 91–94% of the time (cut(L₁, L₂) = 0.094, 0.062); the error shrinks steadily as the small component grows, reaching 0.82 by small = 10 — the exact shape hypothesis 2’s crude form predicts, and the same direction already seen for the disjoint-paths model but only at the larger, sparser $n = 64$ canvas (§4.15). This is not a separate failure: it is the same over-connection already visible in the two-way sweep of §4.2, where cut degraded steadily from 1.000 to 0.149 as the split balanced. Whether the “which component” skill survives a third, disconnected piece is therefore not tied to which training distribution was used — it tracks whether *that particular* checkpoint has an over-connection tendency at all, which this report finds in the ER-trained model at $n = 40$ and in the disjoint-paths model only at the larger $n = 64$ canvas, but not in the disjoint-paths model at $n = 40$ or the ER model at $n = 64$.

small	large ₁	large ₂	reach large ₁	reach large ₂	cut(S,L ₁)	cut(S,L ₂)	cut(L ₁ ,L ₂)
1	19	20	0.979	0.975	1.000	1.000	0.094
2	19	19	0.984	0.987	1.000	1.000	0.062
4	18	18	0.998	0.997	1.000	1.000	0.014
7	16	17	0.955	0.930	0.758	0.775	0.371
8	16	16	0.901	0.900	0.853	0.854	0.619
10	15	15	0.881	0.880	0.887	0.890	0.820

Table 7: *Model/test/metrics as in Table 6 but trained on pure sparse Erdős–Rényi (ER(40, 0.05)), four seeds, $n = 40$.*

4.11 Attention and read-out weights, visualised

The attention pattern itself changes shape between a solved and a failed split. Figure 5 shows the real attention scores $S = QK^\top/\sqrt{d_h}$ and the resulting normalized-ReLU α , per layer, for one example graph at $a = 4$ and one at $a = 20$. Layer 0 is essentially identical in both: a narrow band along the diagonal (each node attending to its immediate path neighbours), broken cleanly at the component boundary — a purely local, structure-driven pattern that does not depend on the split. Layer 1 is where the two conditions diverge. The exact contribution C_{ik} (Figure 6, §2) is concentrated in a band around the diagonal that *decays* with distance

rather than forming a flat, uniform block — at $a = 4$, a pair three-or-fewer hops apart in the long component averages roughly twice the contribution of a pair 15 or more hops apart (53 vs. 25 in the units of Figure 6), and the same ratio holds, at a much smaller overall scale, at $a = 20$ (8.5 vs. 5.8). What changes with the split is where that band is cleanly *cut*: at $a = 20$ it breaks sharply at the component boundary into two separate diagonal blocks, matching the leak-fraction curve of Figure 3; at $a = 4$ no such clean break is visible, because the short component is too small (four nodes) for a break to show up as a distinct block against the long one. One structural feature holds at *every* split: the far endpoint of each path — the lone degree-1 node, structurally distinct from every other node on the path — is an unusually large *source* of contribution to the rest of its component (column sums 3–4 $\times$ the per-node average at $a = 4$ and $a = 20$ alike) without being an unusually large *receiver*, consistent with the endpoint broadcasting a “this path closed” signal outward rather than absorbing one. The overall scale itself also tracks the split: raw contribution values are roughly 6–8 $\times$ larger at $a = 4$ than at $a = 20$ (colour scales in Figure 6 are not shared, on purpose, to keep each panel legible), the same direction as the saturated-vs-faint raw logit magnitudes already seen in §4.1. The row-mass panels (right column, Figure 6; unaffected by any of the above, since row-mass is a property of α alone) make the attention-pattern shape concrete: at $a = 4$, layer-1 row-mass rises once past the component boundary and stays on a flat, high plateau for the whole long path; at $a = 20$, it rises after *each* boundary but dips again in the middle of each half, a double-hump rather than a plateau. Per-node $Q/K/V$ vector norms (not shown in the main figures; computed for every split) are essentially *identical* in shape between $a = 4$ and $a = 20$ — the difference is not in how strongly a node’s query/key/value is scaled, only in which other nodes’ keys and values it lines up with.

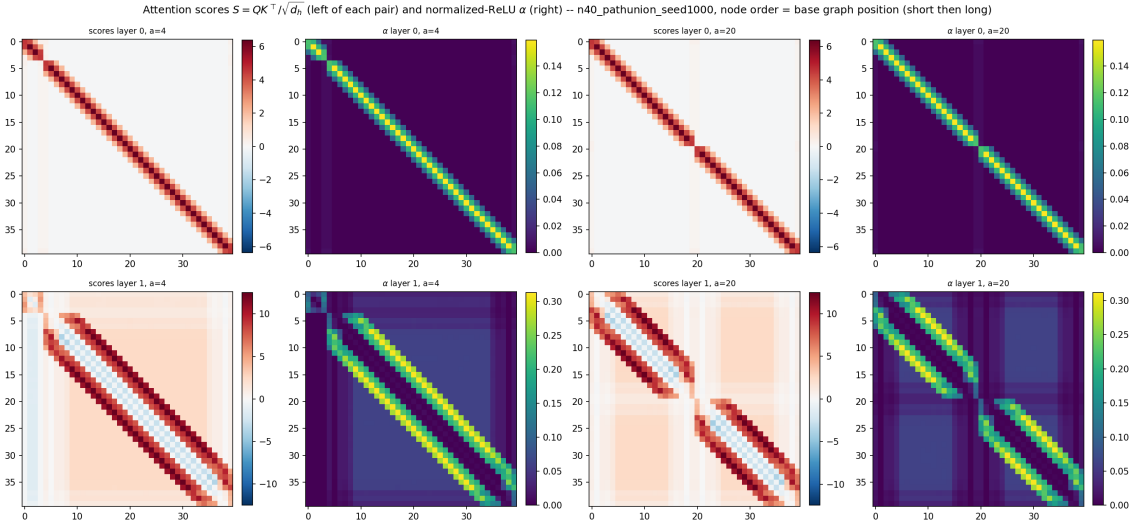


Figure 5: *Model*: one seed of the checkpoints of Table 1 (base standard transformer, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, disjoint-paths-trained, $n = 40$). *Test set*: one representative two-chain graph at $a = 4$ and one at $a = 20$, node order fixed to base graph position (short path first, then long) for readability. *Metric*: the real attention score matrix $S = QK^\top / \sqrt{d_h}$ and the resulting normalized-ReLU attention $\alpha = \text{ReLU}(S)/n$, layer 0 (top) and layer 1 (bottom), averaged over 80 independent random relabellings of the same underlying graph then mapped back to base node order (so the figure reflects the model’s stable behaviour on this graph shape, not one noisy instance).

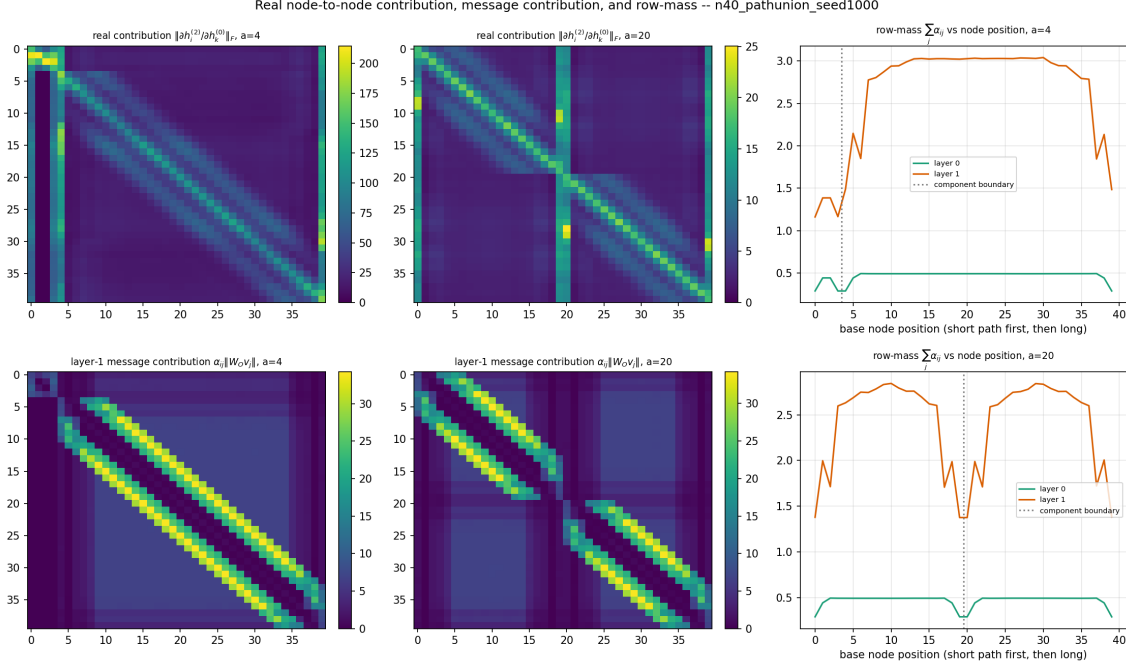


Figure 6: *Model/test as in Figure 5*, 40 independent relabellings averaged for the top row. *Metric*: the exact node-to-node contribution $C_{ik} = \|\partial h_i^{(2)}/\partial h_k^{(0)}\|_F$ (top row, §2; real automatic differentiation through V , W_O , the residual connections and the MLP, not an attention-only approximation) and the layer-1 message contribution $\alpha_{ij}\|W_O v_j\|$ (bottom row) — how much node j ’s value, after the output projection, actually adds to node i ’s residual stream at that one layer — for $a = 4$ (left) and $a = 20$ (middle); and the row-mass $\sum_j \alpha_{ij}$ (right column) along the chain, layer 0 vs. layer 1, for both splits, dotted line marks the component boundary. Colour scales are independent per panel (not shared across $a = 4$ and $a = 20$).

The read-out and read-in weight matrices show no exploitable structure (further evidence against hypothesis 3). Figure 7 renders what Table 4 already summarised in numbers: W_{out} and W_{in} look like unstructured noise at the level of individual entries, the cosine-similarity matrices between different target/label vectors are uniformly small off the diagonal, and the skip-connection alignment $E_{\text{in}} W_{\text{out}}^\top$ has no diagonal or banded pattern beyond a few scattered entries — there is no visible “special” node index, and no sign that the model precomputes an index-based shortcut anywhere in these two matrices. The singular-value spectra of every weight matrix in the trunk (Figure 8) decay smoothly with no isolated dominant direction, consistent with a distributed computation rather than a single low-rank shortcut circuit.

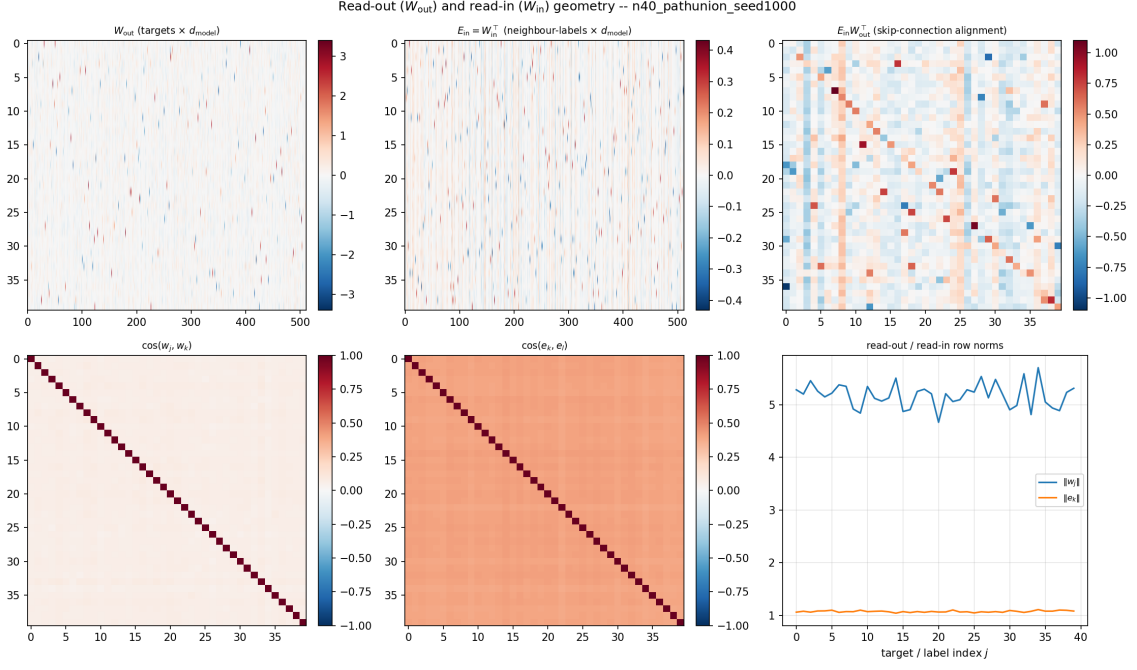


Figure 7: *Model*: the four checkpoints of Table 4 (static weights, no test graphs). *Metric*: W_{out} and $E_{in} = W_{in}^T$ (raw entries); the skip-connection alignment $E_{in} W_{out}^T$; the cosine-similarity matrices $\cos(w_j, w_k)$ and $\cos(e_k, e_l)$; and the row norms $\|w_j\|, \|e_k\|$ by target/label index (one representative seed shown).

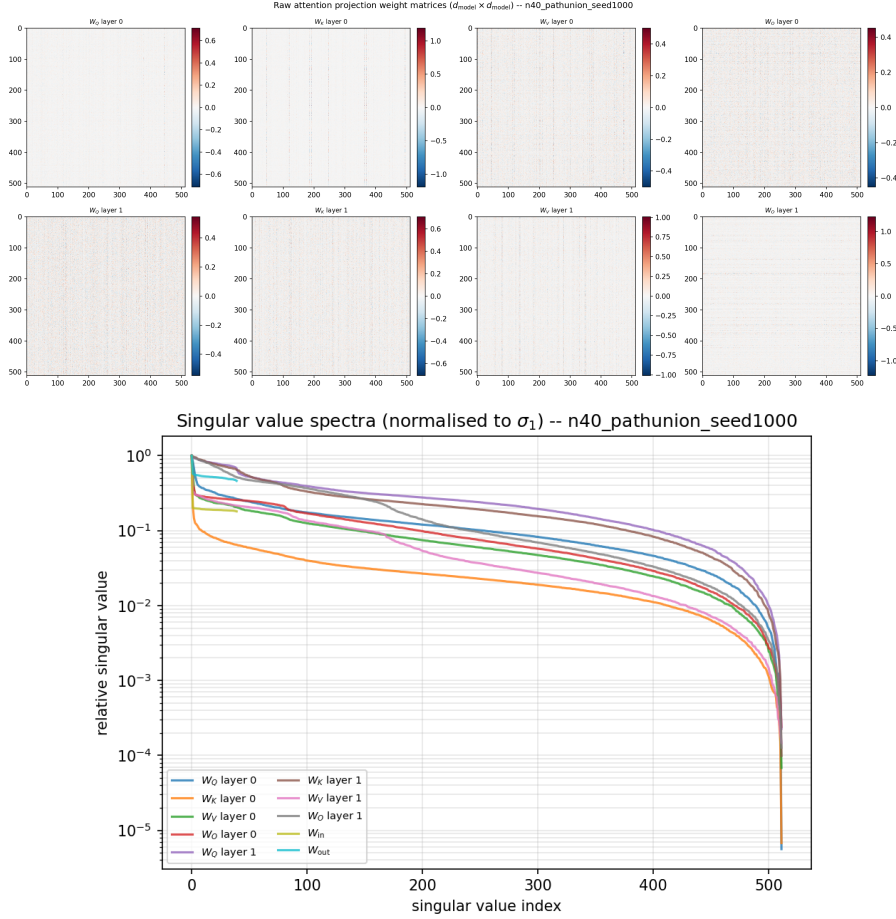


Figure 8: *Model as in Figure 7. Top: the raw attention projection matrices W_Q, W_K, W_V, W_O for both layers ($d_{\text{model}} \times d_{\text{model}}$). Bottom: singular value spectra of every trunk weight matrix, normalised to the largest singular value (one representative seed).*

4.12 The same heatmaps on the ER-trained model: a cleaner cut, no broadcasting endpoint

Figure 9 repeats the exact-contribution heatmap on the ER-trained checkpoints. Two differences from the disjoint-paths model stand out. First, the near-diagonal band is already visibly *cut* at the component boundary at $a = 4$, not only at $a = 20$: a distinct bright block covers the four short-component nodes, separated from the long component’s own diagonal band, where the disjoint-paths model showed no such separation until the short side grew much smaller a fraction of the canvas. Second, the far-endpoint-as-source effect found throughout the disjoint-paths model (§4.11) is *absent* here: the two path endpoints’ contribution-column sums are within a few percent of the per-node average at both $a = 4$ and $a = 20$ (e.g. 105 and 70 against a mean of 105 at $a = 4$), not the $3\text{--}4\times$ elevation seen in the disjoint-paths model. The raw magnitude is also far smaller throughout, consistent with §4.8’s reading of a model that never saturates its read-out logit: near-diagonal contribution is 4.4 at $a = 4$ and 4.4 at $a = 20$ (compare 53 and 8.5 for the disjoint-paths model), a roughly constant scale rather than the $6\text{--}8\times$ swing with split seen there. Both differences point the same way: the disjoint-paths training builds a specific, split-shape- sensitive signal (a broadcasting endpoint, a scale that tracks whether the short side is resolvable) that an ER-trained model, which never saw a path endpoint mean anything, simply does not acquire — it falls back on the same modest, distance-decaying, cleanly-cut local pattern at every split.

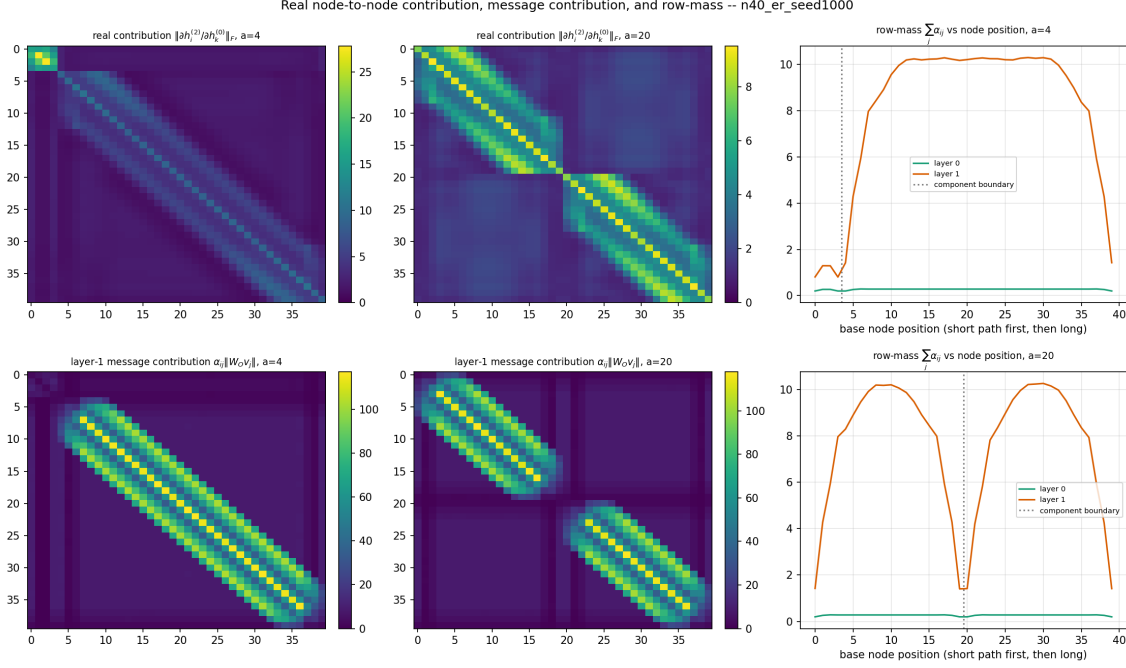


Figure 9: *Model/metric as in Figure 6 but trained on pure sparse Erdős–Rényi (ER(40, 0.05)), 64 independent relabellings averaged for the top row, $n = 40$.*

4.13 Localising the completion signal: layer ablations

Question. Which part of the trunk is responsible for the extra, split-specific completion signal — and is it the same machinery that does ordinary distance-bounded reach, or a separate add-on?

Setup. Starting from an otherwise normal forward pass, we zero one branch of the residual stream at a time — the attention output or the feed-forward output, at layer 0 or layer 1 — and re-run the full split sweep; a fifth condition bypasses both transformer blocks entirely (read-out applied directly to the read-in embedding, no cross-node mixing at all). If ablating a component collapses the $a \leq 7$ behaviour toward the ablated baseline everywhere, that component is doing the work; if the ablated model is essentially unaffected, it is not.

Result: reach and cut depend on different parts of the trunk, and the completion signal is not a separate circuit — it needs the same attention layers as ordinary reach. Figure 10 and Table 8 show a clean split. **Either** attention layer alone is necessary for reach: zeroing layer 0’s attention collapses reach to 0.06–0.11 at *every* split, including $a \leq 7$; zeroing layer 1’s collapses it to 0.15–0.29 — both far below even the ordinary beyond-capacity floor, and both close to the no-mixing-at-all bypass baseline (0.32–0.34, flat in a). The $a \leq 7$ completion signal does not survive losing either attention layer any better than the ordinary $a \geq 11$ regime does: it is built by the *same* two-layer attention machinery, not a separate mechanism riding on top of it. The feed-forward branches tell a different, sharper story. Zeroing layer 1’s feed-forward leaves reach almost untouched (0.87–0.99, barely below baseline at every split) but collapses *cut* from ≈ 1.0 to 0.26–0.62 once the split is no longer tiny — **cut and reach are read out through different final-layer computations**: cut depends heavily on layer 1’s feed-forward step, reach barely does. Layer 0’s feed-forward instead matters more for reach in the completion regime specifically (baseline 1.0 $\rightarrow$ 0.47 at $a = 7$ under ablation, a much larger relative hit than the 0.71 $\rightarrow$ 0.59 it takes at $a = 20$) while cut is only mildly affected throughout.

condition	$a=1$	$a=4$	$a=7$	$a=8$	$a=10$	$a=14$	$a=17$	$a=20$
<i>reach (long component)</i>								
baseline	1.000	1.000	1.000	0.993	0.651	0.581	0.638	0.709
zero attn 0	0.058	0.063	0.068	0.070	0.074	0.084	0.094	0.107
zero attn 1	0.154	0.168	0.182	0.187	0.199	0.226	0.253	0.288
zero mlp 0	0.808	0.635	0.469	0.428	0.438	0.490	0.535	0.593
zero mlp 1	0.992	0.934	0.925	0.916	0.912	0.899	0.885	0.869
bypass	0.322	0.325	0.326	0.328	0.326	0.332	0.332	0.336
<i>cut (target: disconnected)</i>								
baseline	1.000	1.000	0.998	0.926	0.999	1.000	1.000	1.000
zero attn 0	0.995	0.995	0.994	0.994	0.993	0.993	0.993	0.993
zero attn 1	0.986	0.997	0.996	0.996	0.995	0.995	0.993	0.994
zero mlp 0	0.969	0.887	0.898	0.945	0.942	0.917	0.914	0.912
zero mlp 1	0.939	0.599	0.617	0.618	0.534	0.276	0.262	0.256
bypass	0.660	0.692	0.689	0.693	0.688	0.691	0.692	0.691

Table 8: *Model/training as in Table 1*, four seeds. *Test set*: two-chain graphs at the splits shown, 150 graphs per seed, independently node-relabelled. *Metric*: reach (long component, top block) and cut (bottom block), mean over seeds, under six forward-pass conditions — unmodified (baseline); the attention or feed-forward branch zeroed at layer 0 or 1; and both transformer blocks skipped entirely (bypass, read-out applied directly to the read-in embedding).

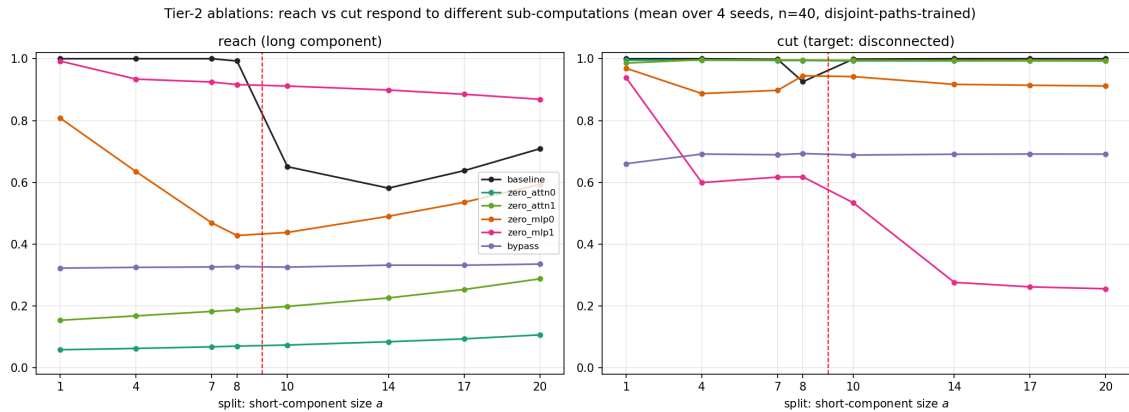


Figure 10: Table 8 as curves. *Model/training/test as in Table 8*, mean over four seeds. Red dashed line: capacity $3^L = 9$.

4.14 The same ablations on the ER-trained model: the double-edged mechanism, already at $n = 40$

Result: zeroing either attention layer collapses reach but *fixes* cut — the over-connection is built by the same machinery as the disjoint-paths model’s completion signal. Table 9 repeats the ablation on the ER-trained checkpoints. Reach depends on both attention layers just as it does for the disjoint-paths model (zeroing layer 0 collapses it to 0.04–0.07, zeroing layer 1 to 0.13–0.25). Cut tells the sharper story: zeroing *either* attention layer pushes cut back up to ≥ 0.996 at every split — fully repairing the degraded baseline (0.15–0.53 once the split is no longer tiny) — while zeroing layer 1’s feed-forward collapses cut further still (down to 0.13 at $a = 20$), the same direction as the disjoint-paths model. This is exactly the double-edged pattern this report finds for the disjoint-paths model only at the larger $n = 64$ canvas (§4.15): the identical two-layer attention computation that lets the model confirm a connection is also what erodes its ability to keep two components apart,

and switching it off trades one failure for the other rather than fixing either independently. Finding it here, in a model trained on a completely different distribution at the *smaller* $n = 40$ canvas, says this is not a 64-node-canvas-specific quirk: it is a general property of how this two-layer attention mechanism trades reach against cut whenever a checkpoint leans toward over-connection, whatever produced that lean.

condition	$a=1$	$a=4$	$a=7$	$a=8$	$a=10$	$a=14$	$a=17$	$a=20$
<i>reach (long component)</i>								
baseline	0.965	0.999	0.977	0.946	0.902	0.925	0.942	0.960
zero attn 0	0.036	0.042	0.044	0.046	0.049	0.055	0.062	0.070
zero attn 1	0.131	0.141	0.154	0.159	0.169	0.194	0.218	0.251
zero mlp 0	0.440	0.511	0.456	0.458	0.465	0.500	0.531	0.577
zero mlp 1	0.994	0.981	0.973	0.972	0.968	0.959	0.949	0.932
bypass	0.231	0.231	0.232	0.232	0.232	0.231	0.231	0.233
<i>cut (target: disconnected)</i>								
baseline	1.000	0.999	0.534	0.558	0.520	0.290	0.192	0.152
zero attn 0	1.000	0.999	0.998	0.997	0.997	0.996	0.996	0.996
zero attn 1	0.996	1.000	1.000	1.000	1.000	1.000	1.000	1.000
zero mlp 0	1.000	0.772	0.884	0.882	0.884	0.876	0.875	0.869
zero mlp 1	0.948	0.507	0.338	0.266	0.174	0.143	0.130	0.129
bypass	0.691	0.753	0.760	0.760	0.763	0.763	0.763	0.763

Table 9: *Model/test/metric as in Table 8 but trained on pure sparse Erdős–Rényi (ER(40, 0.05)), four seeds, $n = 40$.*

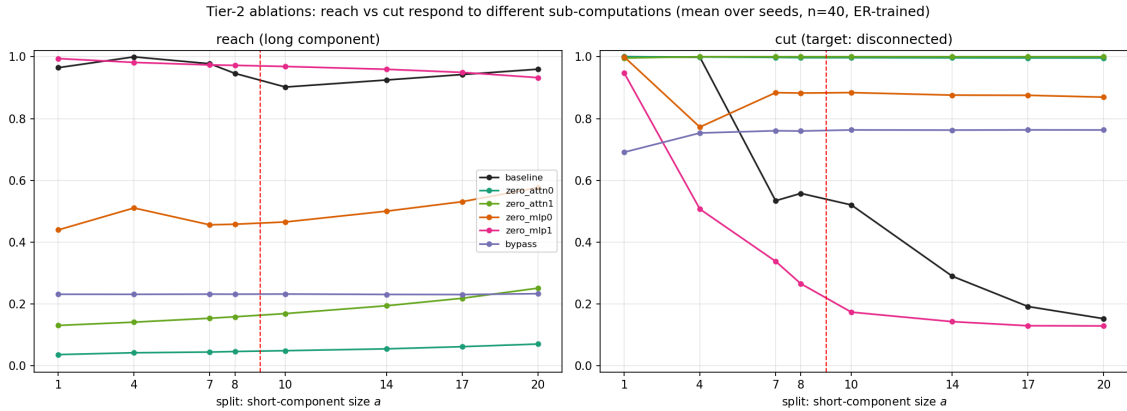


Figure 11: Table 9 as curves. *Model/training/test as in Table 9, mean over four seeds. Red dashed line: capacity $3^L = 9$.*

4.15 Generalising to $n = 64$: path_union and ER

Question. Every result so far is on one canvas size, $n = 40$. Does the same completion signal appear at a larger, sparser canvas, and does it depend on having seen disjoint-path structure in training at all? We repeat the full Tier 1/2 audit on the $n = 64$ checkpoints from Report VI’s Thread A — already trained, no new training needed — for two training distributions: the same disjoint-paths family as everywhere else in this report, and pure sparse Erdős–Rényi (which never contains a multi-path or two-chain graph).

Result: ER shows no completion signal at all, and tracks the distance oracle almost exactly at every split. Figure 12 is the cleanest single result in this report. Trained purely

on ER, the model shows *no* special behaviour at small a : reach climbs smoothly from 0.21 to 0.39 as a goes from 1 to 32, sitting within a few points of the distance- ≤ 9 oracle rate at *every* split, with no cliff, no floor, no recovery — exactly the profile of plain distance-bounded matrix powering, never departing from it. Cut, meanwhile, stays at ≥ 0.997 across the entire sweep, including the fully balanced split. The completion signal found throughout this report is not an architectural inevitability of the trunk — it is something the disjoint-paths training distribution specifically teaches; a model that never saw that structure does not acquire it, at any split.

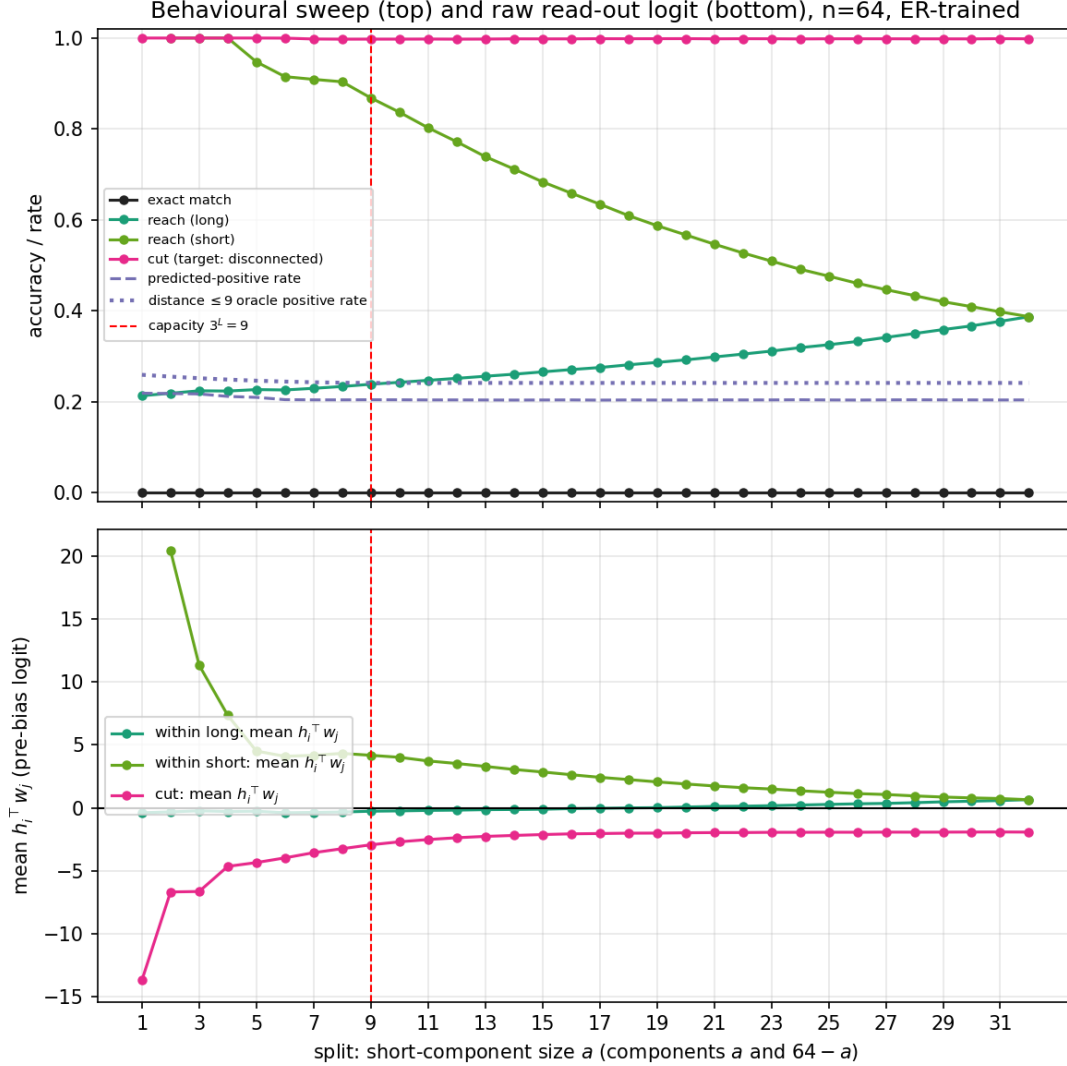


Figure 12: *Model*: base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on pure sparse Erdős-Rényi ($p = 0.03$, mean degree ≈ 1.9 ; never any path structure), four seeds, $n = 64$ (checkpoints from Report VI Thread A). *Test set/metrics as in Figure 1*, full split sweep $a = 1, \dots, 32$.

Result: path_union at $n = 64$ reproduces the completion signal for reach, but the failure mode beyond it inverts — cut collapses instead of staying robust. Figure 13 shows the disjoint-paths-trained $n = 64$ model does still single out small a (exact match is high and reach saturates near 1.0 for $a \leq 4$, tracking the same qualitative shape as $n = 40$), but two things differ from $n = 40$. First, reach *never really collapses*: it decays smoothly from ≈ 1.0 at $a = 1$ to ≈ 0.82 at $a = 32$, always well above the distance oracle, with none of the sharp $n = 40$ -style cliff — on this larger, sparser canvas the model leans toward *over-connecting*

rather than under-connecting once capacity is exceeded. Second, and directly following from that: cut degrades steadily and substantially, from ≈ 0.9 at small a down to ≈ 0.24 by $a = 20$ – 32 — the opposite of $n = 40$, where cut stayed at ≈ 1.0 throughout. The raw read-out logit for cut (bottom panel) makes this precise: it actually *crosses zero* and turns mildly positive past $a \approx 13$, not merely fainter as at $n = 40$ — the model is not just less confident that distant pairs are disconnected, for many of them it now positively believes they are connected. Exact match at very small a is also visibly noisier than the tight $n = 40$ result (per-seed at $a = 4$: 0.43, 0.55, 0.79, 0.00), consistent with Report VI’s own note that the $n = 64$ path_union checkpoints are a noisier, more seed-dependent read than $n = 40$ ’s tight clean model.

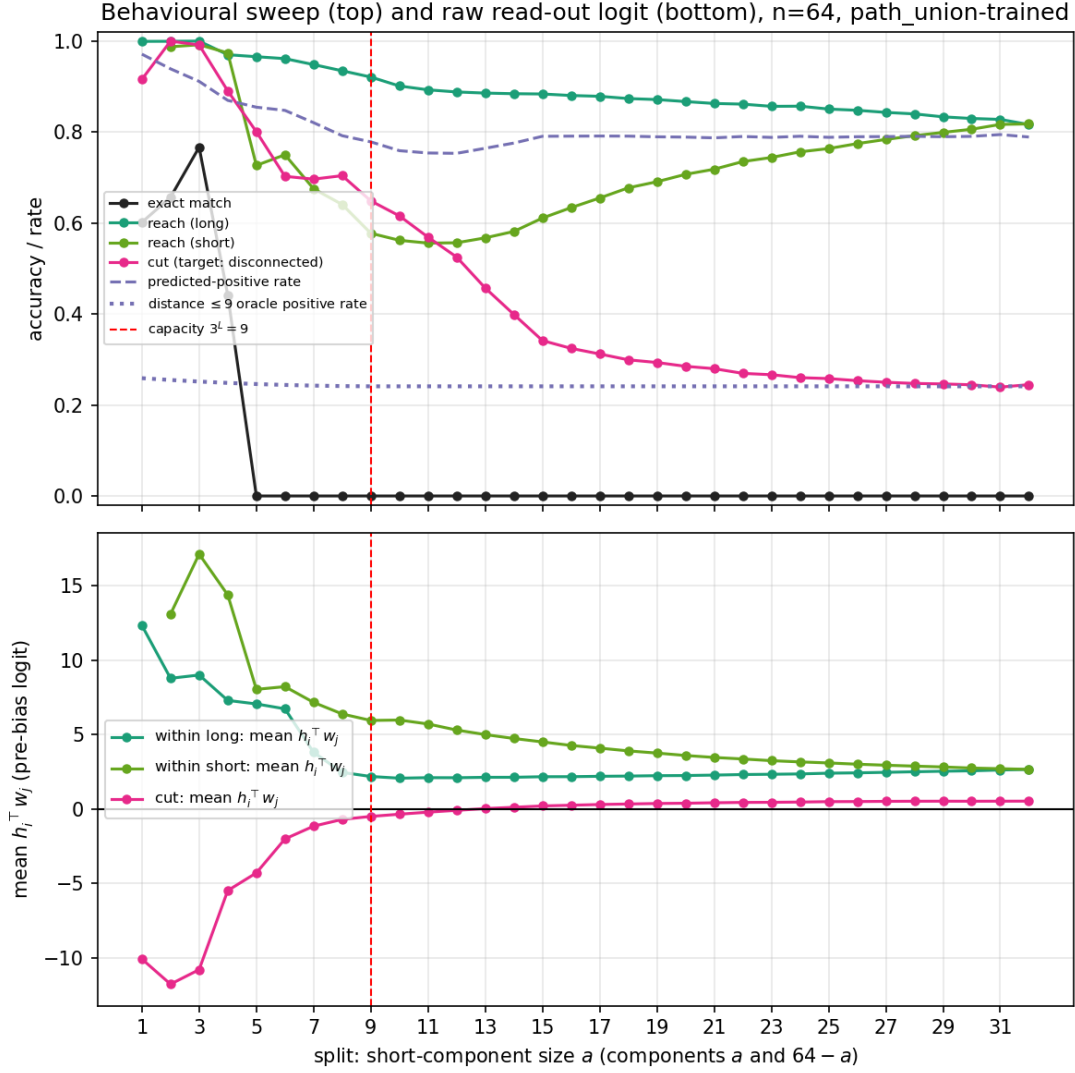


Figure 13: *Model/metrics as in Figure 12 but trained on the same disjoint-paths distribution as every other result in this report, four seeds, $n = 64$ (checkpoints from Report VI Thread A). Full split sweep $a = 1, \dots, 32$.*

The three-way falsification test flips too. Repeating §4.9 at $n = 64$ (Table 10) reproduces the ER-vs-path_union contrast cleanly. The ER-trained model keeps $\text{cut}(L_1, L_2) \geq 0.998$ at every small-component size, exactly as at $n = 40$ — the two large, unrelated components are correctly kept apart regardless of training distribution or canvas size. The path_union-trained model does *not*: at a very small third component (small = 1), it wrongly calls the two large (31- and 32-node) components connected 91% of the time ($\text{cut}(L_1, L_2) = 0.093$) — the crude

“not the small one” shortcut hypothesis 2 explicitly predicted and that $n = 40$ refuted. The error shrinks steadily as the small component grows ($0.09 \rightarrow 0.79$ from small = 1 to 10), the same direction as the two-way sweep’s cut degrading with more structure to keep separate. So hypothesis 2’s crude form, refuted at $n = 40$, is *not* refuted in general — whether the “which component” skill survives a third component depends on canvas size and training distribution, tracking the same over-connection tendency visible in Figure 13.

small	path_union-trained			ER-trained		
	reach L_1/L_2	cut(S,L)	cut(L_1,L_2)	reach L_1/L_2	cut(S,L)	cut(L_1,L_2)
1	0.926	0.999	0.093	0.396	1.000	0.998
2	0.707	1.000	0.369	0.404	1.000	0.998
4	0.756	0.999	0.322	0.417	1.000	0.998
7	0.503	0.997	0.612	0.431	1.000	1.000
8	0.427	0.997	0.717	0.438	0.999	0.999
10	0.394	0.998	0.785	0.453	1.000	1.000

Table 10: *Model/training/test as in Table 6*, $n = 64$, four seeds each. *Metrics*: pairwise reach inside the two large components (mean of L_1, L_2); cut between the small and either large component (mean); cut between the two large components (target disconnected — the decisive column, contrast the two blocks).

Ablation confirms layer-1 attention is a double-edged mechanism at $n = 64$: the same computation that builds the completion signal also builds the over-connection.

Figure 14 repeats §4.13 on the path_union $n = 64$ model. Zeroing layer-1 attention collapses reach just as it does at $n = 40$ (0.05–0.10 at every split) — but it also *fixes* cut, pushing it to ≥ 0.93 at every split, well above the degraded baseline. The same is true, more mildly, of zeroing layer-0 attention (cut steady at ≈ 0.88 , versus baseline’s fall to 0.24). Zeroing layer-1’s feed-forward again collapses cut specifically (down to 0.15 at $a = 32$) while leaving reach largely intact, replicating §4.13 exactly. Put together: the two-layer attention mechanism that builds the beyond-capacity reach signal is the *same* mechanism responsible for the over-connection that erodes cut on this larger canvas — removing it trades one failure for the other rather than fixing either independently.

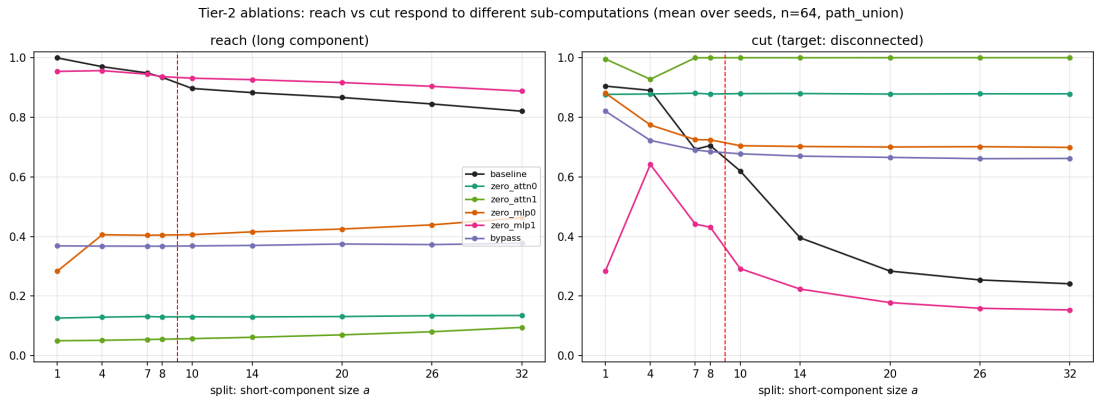


Figure 14: *Model/training/test as in Table 8*, $n = 64$, path_union-trained, four seeds. *Metric as in Figure 10*.

4.16 A third canvas size: $n = 20$, where the capacity wall cannot bite

Question. §4.15 generalised from $n = 40$ to a larger, sparser canvas. At $n = 20$ the same architectural capacity ($3^L = 9$) is close to the whole canvas: even the most balanced split, $(10, 10)$, has a within-component distance that never exceeds 9. So a genuine test of whether this report’s mechanism is specific to $n = 40$ should not expect the same behavioural wall to reappear at $n = 20$ — it is a real, informative check precisely *because* it removes the capacity constraint that drives every other result in this report, not a re-confirmation we expect to look identical.

Result: the disjoint-paths model solves every split, exactly as the capacity argument predicts — but the underlying leak fraction still rises with the split. Table 11 confirms the expectation directly: the disjoint-paths-trained model is exact on essentially every graph at every split from $a = 1$ to the balanced $a = 10$ (≥ 0.98 throughout, reach and cut both 1.000) — there is no wall to expose because nothing tested ever asks it to confirm a pair beyond 9 hops. The interesting result is that the mechanistic signature from §4.7 does not disappear along with the behavioural wall: the leak fraction (Figure 15, left) still rises smoothly from 0.4% at $a = 1$ to 36% at the balanced $a = 10$, essentially the same range as the $n = 40$ result, even though behaviour here is perfect throughout. The completion mechanism therefore is not simply a symptom of the capacity wall being hit; it is a graded property of how the trunk handles a two-component graph that shows up whether or not the split ever forces a visible mistake.

split ($a, 20-a$)	exact	reach (long)	reach (short)	cut	pred. positive rate
(1, 19)	1.000	1.000	–	1.000	0.905
(4, 16)	1.000	1.000	1.000	1.000	0.680
(7, 13)	0.987	1.000	1.000	1.000	0.545
(8, 12)	0.984	1.000	1.000	1.000	0.520
(10, 10)	1.000	1.000	1.000	1.000	0.500

Table 11: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on the same clean disjoint-paths distribution as Table 1, four seeds, $n = 20$. *Test set/metrics as in Table 1*, full sweep $a = 1, \dots, 10$ (selected columns shown).

Result: an ER-trained model at $n = 20$ never solves the two-chain task well at any split, for a reason that is not the capacity wall either. Table 12 shows a genuinely different picture from every other ER result in this report: exact match is low and non-monotone across the whole sweep (peaking at 46% around $a = 3$, not tabulated, and falling to 0 by $a = 6-9$ before a small recovery at $a = 10$), and reach dips in the middle of the sweep (0.77–0.87 at $a = 3-9$) rather than at one end. Since nothing here exceeds the capacity, this is not the distance wall reappearing; it reads instead as a general out-of-distribution sensitivity to two-chain graphs at this canvas size, worst somewhere in the middle of the split range rather than at the balanced point specifically. The leak fraction (Figure 15, right) still rises with the split, from 2% at $a = 1$ to 46% at $a = 10$, the same qualitative direction as every other condition in this report, despite the noisier behavioural backdrop.

split ($a, 20-a$)	exact	reach (long)	reach (short)	cut	pred. positive rate
(1, 19)	0.161	0.932	—	0.995	0.848
(4, 16)	0.176	0.836	0.988	0.990	0.584
(7, 13)	0.000	0.842	0.922	0.904	0.519
(8, 12)	0.000	0.852	0.912	0.887	0.513
(10, 10)	0.060	0.886	0.887	0.868	0.515

Table 12: *Model*: base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on pure sparse Erdős–Rényi (ER(20, 0.08), the paper’s original density), four seeds, $n = 20$. *Test set/metrics as in Table 1*, full sweep $a = 1, \dots, 10$ (selected columns shown).

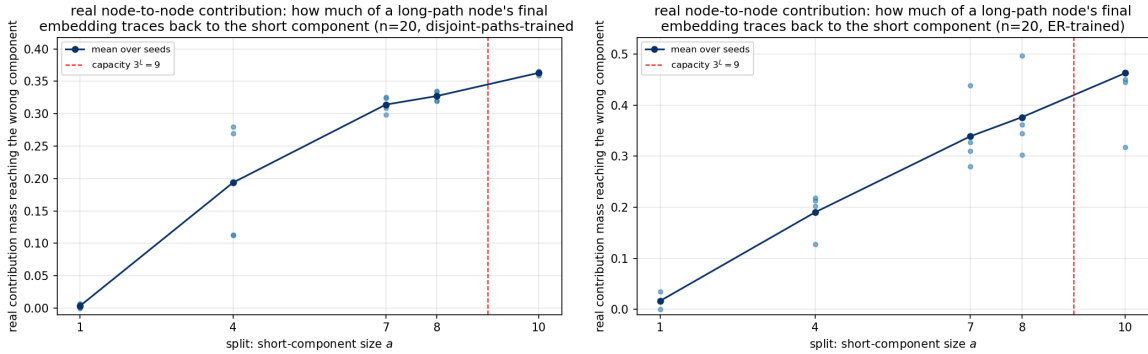


Figure 15: *Model/metric as in Figure 3*, $n = 20$, four seeds, 64 graphs per seed. *Left*: disjoint-paths-trained (Table 11). *Right*: ER-trained (Table 12).

The three-way falsification test is clean for both models here — consistent with over-connection being a beyond-capacity phenomenon. Neither model wrongly merges the two large, disconnected components at $n = 20$: the disjoint-paths model is exact ($\text{cut}(L_1, L_2) = 1.000$ at every small-component size tested), and the ER model is close behind (0.95–1.00). This is the cleanest single confirmation in this report that the crude “not the small one” merge found for the ER model at $n = 40$ (§4.10) and for the disjoint-paths model at $n = 64$ (§4.15) is a beyond-capacity failure mode specifically: remove the possibility of exceeding capacity, and both training distributions keep three components straight.

Ablation: reach is more robust here, and cut depends on more than one component of the trunk. The double-edged pattern from §4.13/§4.14 is present but less clean-cut at $n = 20$. For the disjoint-paths model, reach survives attention ablation far better than at $n = 40$ (zeroing layer 0 leaves reach at 0.40–0.43, zeroing layer 1 at 0.79–0.81, against a baseline of 1.000 — nowhere near the near-total collapse seen at $n = 40/64$), while cut, which is perfect at baseline, is knocked down by *every* ablation tested (0.38–0.85), not specifically by attention. For the ER model, reach again depends heavily on both attention layers (0.11–0.17 and 0.27–0.47 under ablation), and zeroing either attention layer still moves cut in the same repairing direction seen at $n = 40$ (up to 0.99 from a baseline of 0.87–0.99), though the effect is milder since the ER baseline here is already closer to correct. Read together with §4.14, the double-edged reach/cut trade-off in layer-1 attention looks like a general feature of this architecture, not a 64-node or 40-node-specific artefact — it is just harder to see at $n = 20$ because there is less room for either failure mode to develop.

4.17 Does the similarity read-out learn the same thing?

[Results pending — the full battery of §4.1–4.14 is being re-run, eval-only, on the existing similarity-read-out checkpoints (both training distributions, $n = 40$, four seeds each); no data collected yet, nothing below is written from invented numbers.]

5 Verdict

We stated three candidate hypotheses in §1.3. The mechanistic evidence settles them as follows.

Hypothesis 3 (label/order shortcut) — refuted. Fixing node labels to a canonical order instead of relabelling every graph independently changes nothing (Table 3). The read-out matrix shows no target-specific outlier structure, and the direct read-in-to-read-out skip connection carries a signal two orders of magnitude smaller than what actually decides a prediction (Table 4) — whatever the model does, it is not reading node identity off a fixed index mapping.

Hypothesis 2 (short-component-resolved default) — correct in a narrower form than stated, and that form is itself canvas- and training-distribution-dependent. The model does not implement a generic “not in the small component therefore connected” rule: at $n = 40$, the three-way falsification test shows it correctly separates two large, unrelated components from a small one at cut = 1.000 throughout (Table 6), which a crude default-connect rule would get wrong. What survives at $n = 40$ is narrower and, we think, more interesting: on a graph with *exactly two* components, once one of them is small enough that the model can resolve it completely, the *other* component gets an extra, split-specific completion signal — visible as a smoothly saturating positive read-out logit (§4.4) and as attention that stays cleanly confined to the true component (§4.7) — that goes beyond plain distance-bounded reach. That signal is switched off, not merely weakened, once the graph has a third, separate component at $n = 40$: the two large components in §4.9 are each treated exactly like the long side of a balanced two-way split, not like the sole beneficiary of a completion rule. **This clean picture does not fully survive elsewhere in this report**, and not along the line “ER never shows it, disjoint-paths sometimes does” that the $n = 64$ result alone would suggest. At $n = 64$ (§4.15), the disjoint-paths-trained model wrongly merges two large unrelated components when the third is very small (cut(L_1, L_2) = 0.09 at small = 1) while the ER-trained model at the same canvas keeps cut ≥ 0.998 throughout. But at $n = 40$ the roles reverse: the disjoint-paths model is the clean one (Table 6), while an *ER-trained* model shows the crude merge just as badly (cut(L_1, L_2) = 0.01–0.09 at small third components, §4.10). So the crude form of hypothesis 2 is not tied to which training distribution was used; it tracks whether a given checkpoint has an over-connection tendency *at all* on that canvas — something this report finds in the disjoint-paths model only at $n = 64$ and in the ER model only at $n = 40$, matching each model’s own two-way cut degradation (§4.15, §4.2) almost exactly. §4.16 pins down *when* it can happen at all: at $n = 20$, where no split can exceed the architectural capacity, *both* training distributions keep the three components straight (cut(L_1, L_2) ≥ 0.95 throughout). The crude merge is a beyond-capacity failure mode, available to any checkpoint that has learned to over-connect once distance exceeds 3^L , not a property of one architecture or one training distribution.

Hypothesis 1 (genuine, distance-graded propagation) — correct once the completion signal is not active. Beyond the two-component-with-a-small-side regime, the model’s aggregate behaviour is very hard to distinguish from plain distance-bounded matrix powering: the overall predicted-positive rate for $a \gtrsim 11$ sits almost exactly on the rate a pure distance- ≤ 9 oracle would give on the same graphs (§4.1), and the attention-leak curve (Figure 3) is graded and already non-zero at the splits the model solves perfectly, rather than switching on and

off at a threshold. The “genuine propagation” account is the right description of the model *outside* the specific regime where the completion shortcut fires — it is not a competing story to hypothesis 2, it is what the model falls back to when hypothesis 2’s narrow condition is not met.

Put together. The Report VI puzzle is not really about balance. It is about whether a two-component graph contains one component small enough to fully resolve. When it does, the model gets an extra completion signal for the other component that reaches far beyond its ordinary distance-bounded capacity (the 4+36 case); when it does not — whether because the split is balanced, or because a third component is present at all — that signal does not fire, and the model’s behaviour reverts to the same distance-bounded, 3^L -capped propagation documented since Report III.

Where the signal lives. The ablations of §4.13 add a mechanistic layer to that picture rather than a competing one. Reach and cut are computed by different parts of the trunk — cut depends heavily on layer 1’s feed-forward step and barely on either attention layer’s presence *once both are intact*, while reach collapses if *either* attention layer is removed, at every split, including the completion regime. In other words, the $a \leq 7$ completion signal is not a separate sub-circuit bolted onto an otherwise-ordinary matrix-powering computation: it needs the same two-layer attention machinery that ordinary beyond-capacity reach needs, and the attention-score heatmaps (§4.11) show *why* — the same layer-1 attention that forms a distance-decaying band cleanly cut at the component boundary at large a forms the same decaying band *without* that clean cut once the other side is small enough to resolve, because a stub of only a few nodes is too small to register as a separate block. It is the same mechanism doing more, not two mechanisms.

$n = 64$ confirms the mechanism is data-shaped, not architectural, and reveals its dark side. The ER-vs-path_union contrast at $n = 64$ (§4.15) is the cleanest evidence in this report that the completion signal is something specific training distributions teach: the ER-trained model never acquires it, at any split, and tracks a plain distance-bounded oracle almost exactly throughout. The path_union-trained model does acquire it, and at $n = 64$ the ablations show it is a double-edged version of the *same* mechanism found at $n = 40$ — the identical two-layer attention computation that lets a small-side graph confirm connections far beyond capacity also, on a larger and sparser canvas, starts declaring unrelated large components connected to each other. Zeroing layer-1 attention removes both effects together (Figure 14): reach collapses, but cut is fully restored. Reach and cut were already shown to depend on different final-layer computations (cut on layer-1 feed-forward, largely independent of either attention layer’s presence); what $n = 64$ adds is that the attention mechanism which *builds* the useful completion signal is the same one that, past a point, over-generalises it into a harmful one.

The double-edged mechanism is general; one specific piece of it is not. The ER-trained model at $n = 40$ (§4.14) shows the identical double-edged ablation signature found at $n = 64$ — zeroing either attention layer restores its already badly-degraded cut to ≥ 0.996 while collapsing reach — so this trade-off is not a 64-node-canvas-specific curiosity; it appears wherever a checkpoint has developed an over-connection tendency, which §4.16 shows requires exceeding capacity in the first place. One piece of the $n = 40$ disjoint-paths mechanism does *not* generalise this way: the far path-endpoint’s role as an outsized *source* of contribution (§4.11) is absent in the ER-trained model at the same canvas (§4.12), whose endpoints contribute at the same level as every other node. A model that never saw a path end mean anything in training does not single that position out — so this specific broadcasting signal, unlike the general

reach/cut trade-off in layer-1 attention, is something the disjoint-paths distribution specifically teaches, consistent with how this report reads the completion signal as a whole.

6 Honest limits

This report opens the trunk further than any before it, but the evidence remains statistical and correlational, not a proof of what the network computes. The read-out, attention, and ablation analyses show quantities and sub-computations that move in lockstep with behaviour; they do not amount to a formal account of *why* training produces a completion signal specific to a two-component canvas. A causal intervention — transplanting activations between a solved and a failing graph, or intervening on attention patterns directly — is left for future work. Two further follow-ups were not run in this round and are left open: a retrain with a loss that reweights within-component and cross-component pairs evenly, to separate a genuine representational effect from a training-objective artefact (the confound Report VI’s own outlook names); and the same audit repeated with the similarity read-out, whose logit is a direct function of *both* endpoints’ embeddings and may not show the same asymmetric, target-specific completion signal at all. We have not checked whether the two-component completion signal survives under the mixed training stream of earlier reports — the $n = 64$ generalisation (§4.15) checks canvas size and a from-scratch training distribution, not that stream specifically.